Penguin Species Classification with a PyTorch MLP

Wacharawit Chunhahirun | Student ID 68070312 | Fundamentals of Deep Learning | 30 October 2026

1. Problem and Data

Task. Given four body measurements (bill length, bill depth, flipper length, body mass) and two categorical attributes (island, sex) of one penguin, predict its species: Adelie, Chinstrap or Gentoo. This is single-label multi-class classification on a tabular feature vector. The dataset was chosen because it is real field data with missing values and categorical columns, so data handling matters, while being small enough to iterate quickly and to inspect individual errors.

Source and size. Palmer Penguins (Horst, Hill and Gorman, 2020), measurements collected at Palmer Station, Antarctica (Gorman et al., 2014), loaded with `seaborn.load_dataset('penguins')`. The file has 344 rows; 11 rows with missing values were dropped, leaving **333** examples. The 9 model inputs are the 4 numeric columns plus 5 one-hot columns (island, sex).

Split. A random 60/20/20 split with a fixed seed (0): 201 train, 66 validation, 66 test. The split is random, not stratified, so class proportions differ slightly between sets (table below). Numeric features are standardized with a scaler **fit on the training split only** and then applied to validation and test, which prevents leakage. The test set is evaluated once, after the configuration is fixed.

Species	Train	Validation	Test	Total
Adelie	85	23	38	146
Chinstrap	41	17	10	68
Gentoo	75	26	18	119
Total	201	66	66	333

Table 1. Class counts per split.

2. Architecture and Training Choices

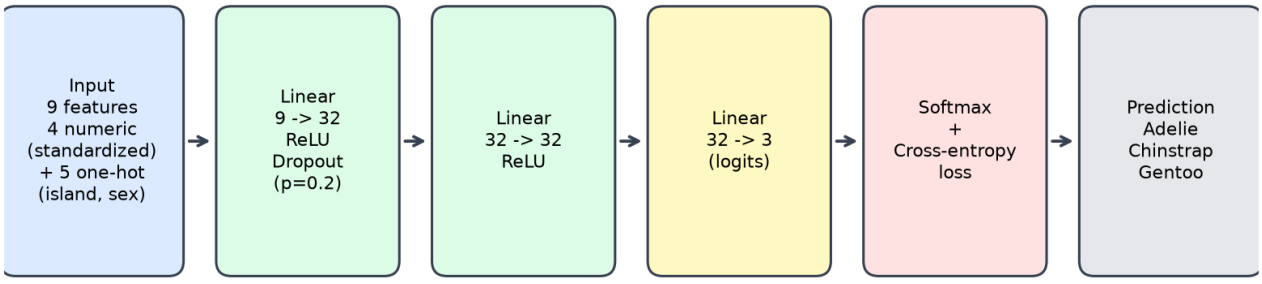


Figure 1. Network architecture and data flow.

Model family. Each penguin is a fixed-length vector of 9 unordered features, with no spatial or sequential structure for a CNN, RNN or transformer to exploit. A small multilayer perceptron (MLP) is the natural neural model for this input. With only 201 training examples, capacity must stay small: the network has two hidden layers of 32 units and **1,475** trainable parameters. It is trained from random initialization; no pretrained weights are used.

Loss. Cross-entropy on the three logits (softmax followed by negative log-likelihood). The classes are mutually exclusive, so a single softmax distribution is appropriate, and cross-entropy is a proper scoring rule that rewards calibrated probabilities, not only the right argmax.

Optimizer. Adam with learning rate 0.001, batch size 16, 8 epochs. Adam adapts step sizes per parameter and works well on small networks with little tuning. Note that `weight_decay` in `torch.optim.Adam` is

coupled L2 regularization, not the decoupled decay of AdamW.

Regularization. (i) Dropout (p = 0.2) after the first hidden layer only, so the second hidden layer and the output layer are not perturbed; (ii) L2 weight decay of 0.0001; (iii) a deliberately small network. Standardizing the numeric inputs is not regularization but keeps features on comparable scales (body mass is in thousands of grams, bill depth in tens of millimetres), which matters for gradient-based training.

Setting	Value
Inputs	9 (4 standardized numeric + 5 one-hot)
Hidden layers	2 x 32 units, ReLU
Dropout / weight decay	0.2 (after layer 1) / 0.0001
Optimizer / learning rate	Adam / 0.001
Batch size / epochs	16 / 8
Parameters	1,475
Seed / hardware / PyTorch	0 / CPU / 2.13.0+cpu

Table 2. Final configuration. This report does not include a systematic hyper-parameter search (see Section 6).

3. Evaluation Protocol

Metric. The primary metric is classification **accuracy** on the held-out test set, supplemented by test cross-entropy loss, per-class recall and the confusion matrix. Accuracy alone can hide failures on the smaller classes, so per-class recall is reported next to it.

Baselines. (1) *Majority class*: always predict the most common training class (Adelie); this is the floor any useful model must beat. (2) *Logistic regression* on the same standardized features and the same split; this is the key comparison, because if a linear model does as well, the neural network is not adding value.

Protocol. The validation set is used for inspecting training curves. The test set is evaluated once for the main run. As a robustness check, the whole pipeline is repeated for 5 seeds; each seed re-draws the split *and* the initialization. These repeats are for measuring variability, not for selecting a model.

4. Results

Model	Val accuracy	Test accuracy	Test loss
Majority class (Adelie)	34.8%	57.6%	n/a
Logistic regression	97.0%	98.5%	0.037
MLP (this work)	89.4%	90.9%	0.241

Table 3. Main run (seed 0). Test accuracy 95% Wilson interval for the MLP: [81.6%, 95.8%].

Main result. The MLP reached **90.9%** test accuracy (loss 0.241), against 57.6% for the majority-class baseline and 98.5% for logistic regression. Logistic regression classified 5 more test penguins correctly than the MLP, so the simpler model was at least as good on this split. With 66 test examples, one additional error moves accuracy by 1.5 points and the 95% interval is 14 points wide, so small differences between models should not be treated as real.

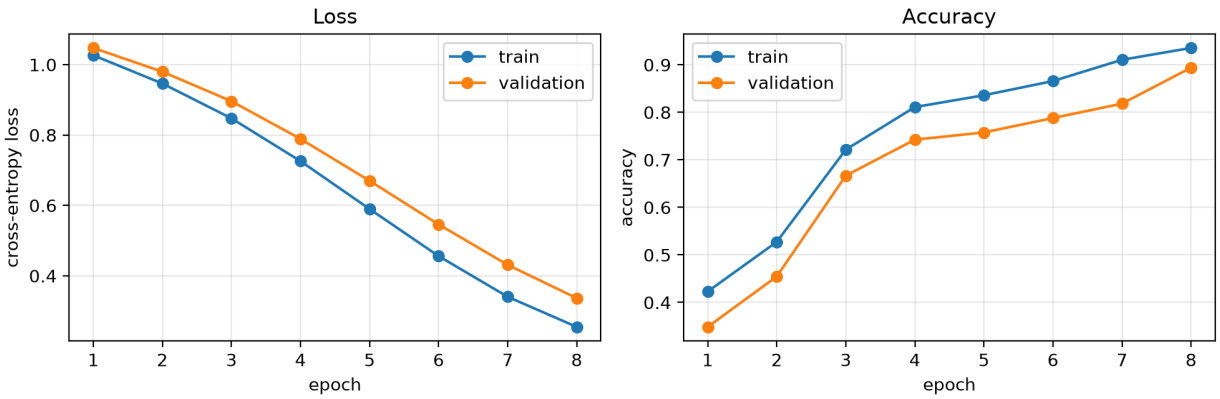
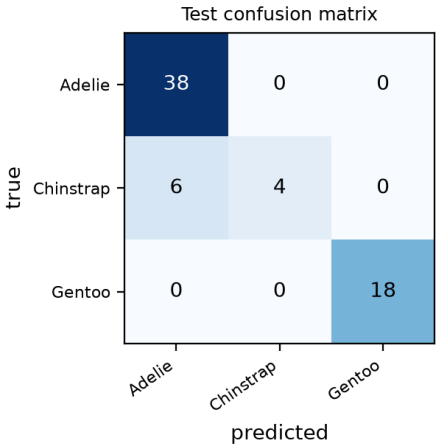


Figure 2. Learning curves (main run). Training metrics are measured in evaluation mode after each epoch.

Training accuracy moved from 42.3% after epoch 1 to 93.5% after epoch 8; validation accuracy moved from 34.8% to 89.4% (best 89.4%, first reached at epoch 8). Final losses were 0.255 (train) and 0.336 (validation). Validation loss was still falling over the last three epochs (by 0.210), so the model had not fully converged after 8 epochs. The gap between training and validation is small, which points to **under-training rather than over-fitting**; the reported test numbers are therefore probably slightly conservative. The validation set has only 66 examples, so single-epoch wiggles of one or two examples (1.5 points each) should not be over-interpreted. Training metrics are computed in evaluation mode (dropout off) after each epoch.



Species	Test n	Correct	Recall
Adelie	38	38	100.0%
Chinstrap	10	4	40.0%
Gentoo	18	18	100.0%

Figure 3 and Table 4. Test confusion matrix and per-class recall (main run).

Per-class recall is lowest for Chinstrap (4 of 10 test examples). Because that class has only 10 test examples, a single error changes its recall by 10 points, so per-class figures are even noisier than overall accuracy.

Seed	0	1	2	3	4	Mean +/- std
MLP test acc.	90.9%	100.0%	92.4%	87.9%	89.4%	92.1% +/- 4.7
Logistic reg. test acc.	98.5%	100.0%	98.5%	98.5%	100.0%	99.1% +/- 0.8

Table 5. Robustness over seeds (each seed changes the split and the initialization).

Across 5 seeds the MLP's test accuracy ranged from 87.9% to 100.0% (mean 92.1%), and logistic regression's from 98.5% to 100.0% (mean 99.1%). The spread shows how much a single random split can flatter or penalize a model, so the headline number in Table 3 should be read together with Table 5.

5. Failure Analysis

The main run made 6 test errors. The cases below are the most confident held-out mistakes.

#	Split	True	Pred.	P(pred)	P(true)	Bill len	Bill dep	Flipper	Mass	Island	Sex
1	test	Chinstrap	Adelie	0.72	0.22	42.5	17.3	187	3350	Dream	Female
2	test	Chinstrap	Adelie	0.60	0.33	45.4	18.7	188	3525	Dream	Female
3	test	Chinstrap	Adelie	0.50	0.38	45.9	17.1	190	3575	Dream	Female

Table 6. Raw feature values of the cases analysed (mm and g).

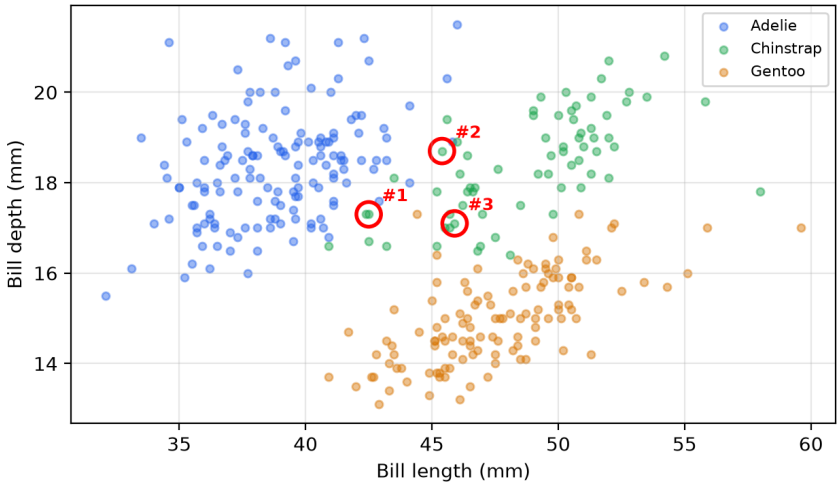


Figure 4. Bill length vs. bill depth for all penguins (coloured by true species); the analysed cases are circled.

Case #1 (test set). A Chinstrap penguin was classified as Adelie with probability 0.72 (probability of the true class: 0.22). The model was only moderately sure, so a confidence threshold could have flagged this prediction for review. In standardized numeric feature space it lies closer to the Adelie class centroid (distance 0.99) than to the Chinstrap centroid (1.68), so it is an atypical Chinstrap, not a random mistake. Its most atypical measurement for a Chinstrap is bill length (42.5 mm, which is 1.2 training standard deviations from the class mean). The class has 41 training examples.

Case #2 (test set). A Chinstrap penguin was classified as Adelie with probability 0.60 (probability of the true class: 0.33). The model was only moderately sure, so a confidence threshold could have flagged this prediction for review. In standardized numeric feature space it is still closer to its own class centroid (1.04 vs 1.23), so simple centroid distance does *not* explain the error; the decision boundary learned from the small training set is the more likely cause. Its most atypical measurement for a Chinstrap is bill length (45.4 mm, which is 0.7 training standard deviations from the class mean). The class has 41 training examples.

Case #3 (test set). A Chinstrap penguin was classified as Adelie with probability 0.50 (probability of the true class: 0.38). The model was only moderately sure, so a confidence threshold could have flagged this prediction for review. In standardized numeric feature space it is still closer to its own class centroid (1.15 vs 1.43), so simple centroid distance does *not* explain the error; the decision boundary learned from the small training set is the more likely cause. Its most atypical measurement for a Chinstrap is bill depth (17.1 mm, which is 0.8 training standard deviations from the class mean). The class has 41 training examples.

What this does and does not show. These are individual examples. They show where the model's boundary is weakest on this split; they do not show why in general. Explanations such as 'overlapping species in bill measurements' or 'too few examples of the minority class' are hypotheses consistent with the numbers above and were *not* tested by an ablation here.

6. Limitations and Next Steps

Limitations.

• **Small data and small test set.** 66 test examples give a wide uncertainty band (95% interval [81.6%, 95.8%]); the per-class numbers are noisier still. Model differences of one or two examples are not meaningful.

• **Single dataset, single population.** All penguins come from three islands and one study. Nothing here shows the model would transfer to other colonies or years.

• **No systematic hyper-parameter search or ablations.** Dropout, weight decay, width, depth and epoch count were fixed, not swept, so this report cannot say which design choices matter.

• **Non-stratified split.** Class proportions differ across splits, which adds variance, especially for the smallest class.

• **Rows with missing values were dropped**, not imputed. If missingness is related to species or measurement conditions, this introduces bias.

• **Calibration was not evaluated.** Only accuracy, loss and recall were reported.

Next steps.

• Use stratified k-fold cross-validation (with the scaler fit inside each fold) for tighter estimates, and report mean and standard deviation instead of a single split.

• Add early stopping on validation loss and a small, logged hyper-parameter search; ablate dropout and weight decay to test which regularizer helps.

• Compare against a gradient-boosted tree model: on small tabular data tree ensembles are often at least as strong as neural networks (Grinsztajn et al., 2022).

• Measure calibration and test a confidence threshold that sends low-confidence predictions to a human, which would address the confident-error problem seen in Section 5.

• Impute missing values rather than dropping rows, and check robustness to that choice.

References

Horst, A. M., Hill, A. P., Gorman, K. B. (2020). palmerpenguins: Palmer Archipelago (Antarctica) penguin data. R package version 0.1.0.

Gorman, K. B., Williams, T. D., Fraser, W. R. (2014). Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins. PLoS ONE 9(3): e90081.

Kingma, D. P., Ba, J. (2015). Adam: A method for stochastic optimization. ICLR.

Srivastava, N. et al. (2014). Dropout: a simple way to prevent neural networks from overfitting. JMLR 15, 1929-1958.

Grinsztajn, L., Oyallon, E., Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? NeurIPS Datasets and Benchmarks.